

Relatório técnico e funcional - FinTrackApp

Data: 23/05/2026

Projeto analisado: FinTrackAppExpo / FinTrackApp

Tipo: aplicativo mobile de controle financeiro pessoal com backend próprio para autenticação e API de dados.

1. Visão geral

O FinTrackApp é um aplicativo de finanças pessoais desenvolvido com **Expo + React Native** no frontend e **Node.js + Express + MongoDB/Mongoose** no backend. A experiência principal é mobile, em orientação retrato, com tema escuro, navegação por abas inferiores e fluxos separados para autenticação, início, saldo, análises, transações e perfil.

O aplicativo já possui autenticação real via API própria, persistência local de sessão com AsyncStorage e rotas protegidas no backend por JWT. A parte financeira está parcialmente integrada: o frontend consome endpoints autenticados de transações, contas e categorias, mas o backend mantém transações e contas em memória e categorias fixas, ou seja, os dados financeiros ainda não estão persistidos em banco.

2. Tecnologias usadas

Carnada	Tecnologia	Uso no projeto
Aplicativo mobile	Expo ~54.0.33	Runtime, build e execução do app React Native.
UI mobile	React Native 0.81.5	Construção das telas, componentes, listas, formulários e estilos.
Linguagem do app	TypeScript ~5.9.2	Tipagem do frontend, navegação e contratos de dados.
Framework de UI	React 19.1.0	Componentização, estado, contexto e ciclo de vida.
Navegação	React Navigation 7	Stack navigation e bottom tabs.
Ícones	@expo/vector-icons / react-native-vector-icons	Ícones Ionicons e MaterialCommunityIcons usados nas telas.
Persistência local	@react-native-async-storage/async-storage	Armazenamento de usuário, access token e refresh token no dispositivo.
Backend	Node.js + Express 4	Servidor HTTP, API REST, middlewares e rotas.
Banco de dados	MongoDB + Mongoose 7	Persistência de usuários e autenticação.
Autenticação	JWT + bcryptjs	Access token, refresh token, hash de senha e sessão.
Segurança API	express-rate-limit + express-validator	Limite de tentativas de login e validação de payload.
Configuração	dotenv	Variáveis de ambiente do backend.
Qualidade	ESLint / eslint-config-expo	Lint do projeto Expo.

3. Estrutura do projeto

```
FinTrackApp/  
+- App.tsx
```

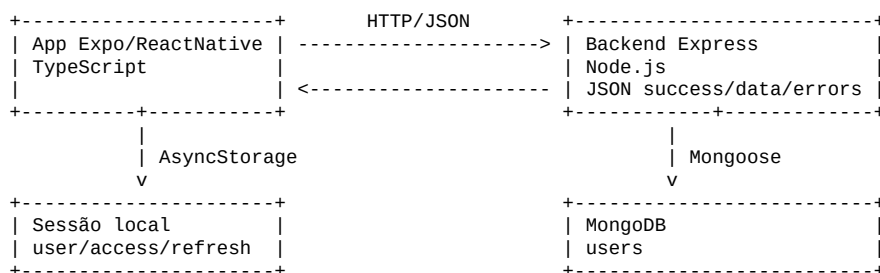
```
# Bootstrap do app, providers e navegação raiz
```

```

+- app.json                # Configuração Expo
+- package.json            # Dependências e scripts do frontend
+- app/
| +- components/common/    # Componentes reutilizáveis de UI
| +- context/AuthContext.tsx # Estado global de autenticação
| +- navigation/           # Stacks, abas e tipos de navegação
| +- screens/              # Telas do aplicativo
| +- theme/index.ts        # Cores, espaçamentos, tipografia e sombras
| +- utils/                # API, storage e mockData
+- backend/
| +- package.json          # Dependências e scripts do backend
| +- src/
| | +- config/database.js   # Conexão MongoDB
| | +- middleware/auth.js   # Proteção por Bearer token
| | +- models/User.js       # Schema de usuário
| | +- routes/auth.js       # Endpoints de autenticação
| | +- routes/data.js       # Endpoints de dados financeiros
| | +- utils/jwt.js         # Geração e verificação JWT
| | +- server.js            # Entrada do servidor Express
+- assets/images/          # Ícones, splash e imagens Expo

```

4. Arquitetura geral



O app decide a rota inicial em `RootNavigator`: se existe usuário autenticado, mostra as abas principais; se não existe, mostra o fluxo de login/cadastro. O contexto de autenticação tenta restaurar a sessão local, valida o usuário com `/auth/me` e, se necessário, renova tokens com `/auth/refresh`.

5. Frontend

5.1 Framework e linguagem

O frontend usa **Expo**, **React Native** e **TypeScript**. O arquivo `App.tsx` configura:

- `GestureHandlerRootView` para gestos.
- `SafeAreaProvider` para áreas seguras em iOS/Android.
- `NavigationContainer` para navegação.
- `AuthProvider` para estado global de autenticação.
- `StatusBar` com estilo claro.
- `SplashScreen` controlada manualmente durante a inicialização.

5.2 Design system

O tema fica em `app/theme/index.ts` e define uma identidade visual escura:

Elemento	Definição
Fundo principal	#0A0F1E
Cards	#1E2A3A, #243040
Acento	Verde #4FFF80

Elemento	Definição
Feedback financeiro	Verde para entradas/economia, vermelho para saídas, azul/âmbar/roxo para categorias.
Tipografia	Escalas de h1, h2, body, caption, label.
Espaçamento	Escala de 4 a 32 px.
Bordas	Radius de 8 a 24 px e full pill.

5.3 Componentes reutilizáveis

Componente	Função
AmountText	Exibe valores monetários com cor/estilo conforme entrada ou saída.
Avatar	Mostra iniciais ou avatar do usuário.
Badge	Indicadores pequenos de status/categoria.
Card	Contêiner visual com tema do app.
Header	Cabeçalho de tela com título e ações.
ScreenWrapper	Wrapper padrão com safe area, scroll e fundo.
SectionTitle	Título de seção.
TransactionItem	Linha de transação com ícone, categoria, data e valor.

5.4 Navegação

```

RootNavigator
+- AuthNavigator
| +- Login
| +- Register
+- MainTabNavigator
  +- Home
  | +- HomeMain
  | +- Categories
  | +- CategoryDetail
  +- Balance
  +- Analytics
  +- Transactions
  | +- TransactionsMain
  | +- AddTransaction
  +- Profile
    +- ProfileMain
    +- EditProfile
    +- Security
    +- AppSettings
    +- Help

```

As abas inferiores têm os rótulos: **Início**, **Saldo**, **Análises**, **Transações** e **Perfil**. A aba ativa usa o verde de destaque e um pequeno indicador superior.

6. Backend

6.1 Framework, linguagem e banco

O backend usa **JavaScript em Node.js**, com **Express** para API REST e **Mongoose** para acesso ao MongoDB. A entrada é backend/src/server.js, que aplica CORS, JSON body parser, registra rotas e só inicia o servidor após conectar ao banco.

6.2 Endpoints de autenticação

Método	Rota	Protegida	Finalidade
GET	/	Não	Health check simples do backend.
POST	/auth/register	Não	Cria usuário local com nome, e-mail e senha.
POST	/auth/login	Não	Autentica e retorna usuário, access token e refresh token.
POST	/auth/social-login	Não	Cria/recupera conta social por provider/providerId.
POST	/auth/refresh	Não	Rotaciona refresh token e emite novo access token.
POST	/auth/logout	Sim	Remove o refresh token do dispositivo atual.
POST	/auth/logout-all	Sim	Remove todos os refresh tokens do usuário.
GET	/auth/me	Sim	Retorna o usuário autenticado.

6.3 Endpoints financeiros

Método	Rota	Protegida	Persistência atual	Finalidade
GET	/data/transactions	Sim	Memória	Lista transações.
POST	/data/transactions	Sim	Memória	Cria uma nova transação.
GET	/data/accounts	Sim	Memória	Lista contas.
GET	/data/categories	Sim	Constante em código	Lista categorias.
GET	/data/summary	Sim	Calculado em memória	Retorna entradas, saídas, saldo e contagens.

6.4 Modelo de usuário

O schema User possui:

- name, email, password para login local.
- provider e providerId para login social.
- avatar, isEmailVerified, tokens de verificação e reset.
- refreshTokens para suportar múltiplos dispositivos.
- isActive, lastLoginAt, createdAt e updatedAt.

A senha é hashada com bcrypt antes de salvar. Campos sensíveis são removidos no toJSON.

7. API no frontend

O arquivo app/utils/api.ts centraliza chamadas HTTP. A URL atual está fixa como:

```
export const API_BASE_URL = `http://192.168.1.70:4000`;
```

Também existe uma função getDebugHost, mas ela não está sendo usada na constante final. Para desenvolvimento em múltiplos computadores, emuladores e redes, o ideal é trocar essa URL fixa por variável de

ambiente ou lógica baseada no host do Expo.

Chamadas principais:

- loginWithEmail
- registerWithEmail
- logout
- refreshAccessToken
- getCurrentUser
- getTransactions
- createTransaction
- getAccounts
- getCategories
- authFetch com renovação automática do token em caso de 401.

8. Estado atual dos dados

Área	Estado atual
Usuários	Persistidos no MongoDB.
Senhas	Hash bcrypt no backend.
Sessão mobile	Persistida no AsyncStorage.
Tokens	Access token e refresh token com rotação.
Transações	Array em memória no backend; mockData também existe no frontend.
Contas	Array vazio em memória no backend; mockData existe no frontend.
Categorias	Lista fixa no backend e mockData no frontend.
Resumo financeiro	Calculado a partir das transações em memória.

9. Telas e mockups

Os mockups abaixo são wireframes textuais baseados nos componentes e textos encontrados no código. Eles representam estrutura, hierarquia e ações esperadas de cada tela.

9.1 Login

Arquivo: app/screens/Auth/LoginScreen.tsx

Objetivo: autenticar usuário existente e permitir navegação para cadastro.

```
+-----+
|               [wallet]               |
|               FinTrack                |
| Controle total do seu dinheiro        |
+-----+
| Bem-vindo de volta                    |
| Entre na sua conta para continuar     |
|                                       |
| E-mail                               |
| [ seu@email.com                      ] |
| Senha                               |
|                                       |
+-----+
```

```

| [ ----- ] |
|               Esqueceu a senha? |
| [ Entrar ] |
|               ou continue com |
| [ Google ] [ Facebook ] |
| Não tem conta? Cadastre-se |
+-----+

```

9.2 Cadastro

Arquivo: app/screens/Auth/RegisterScreen.tsx

Objetivo: criar conta local com nome, e-mail, senha e confirmação.

```

+-----+
| [wallet] |
| FinTrack |
| Cadastre-se e comece a controlar |
| seu dinheiro |
+-----+
| Crie sua conta |
| Preencha os dados abaixo |
| Nome completo |
| [ Seu nome ] |
| E-mail |
| [ seu@email.com ] |
| Senha |
| [ ----- ] |
| Força da senha [barra] |
| Confirmar senha |
| [ Repita a senha ] |
| [ Cadastrar ] |
| Já tem uma conta? Entrar |
+-----+

```

9.3 Início

Arquivo: app/screens/Home/HomeScreen.tsx

Objetivo: apresentar visão rápida do saldo, entradas, saídas e transações recentes.

```

+-----+
| Bom dia, [Nome] [avatar] |
+-----+
| Saldo disponível |
| R$ 12.847,50 |
| +3,2% este mês |
+-----+
| [ Entradas ] [ Saídas ] |
| R$ 4.200,00 R$ 1.832,00 |
+-----+
| Categorias / atalhos |
| [Mercado] [Transporte] [Saúde] |
+-----+
| Recentes Ver todas |
| - Salário +R$ ... |
| - Mercado Extra -R$ ... |
| - Uber -R$ ... |
+-----+
| Início | Saldo | Análises | Trans. |

```

9.4 Categorias

Arquivo: app/screens/Categories/CategoriesScreen.tsx

Objetivo: listar categorias de gastos, total do mês e distribuição percentual.

```

+-----+

```

← Categorias			
TOTAL GASTO EM MAI			
R\$ 1.832,00			
Mercado	R\$ 680	37%	>
[barra de progresso]			
Transporte	R\$ 410	22%	>
[barra de progresso]			
Alimentação	R\$ 328	18%	>
[barra de progresso]			
Entretenimento	R\$ 205	11%	>
Saúde	R\$ 120	7%	>
Presentes	R\$ 89	5%	>

9.5 Detalhe da categoria

Arquivo: app/screens/Categories/CategoryDetailScreen.tsx

Objetivo: exibir detalhes de uma categoria selecionada.

← [Categoria]	
[icone]	
Total gasto	
R\$ xxx,xx	
Transações	[quantidade]
Percentual	[xx%]
Orçamento	[R\$ xxx,xx]
Progresso do orçamento	
[#####.....]	

9.6 Saldo

Arquivo: app/screens/Balance/BalanceScreen.tsx

Objetivo: consolidar saldo total e listar contas financeiras.

Saldo	
SALDO TOTAL	
R\$ 12.847,50	
+R\$1.360 economizados este mês	
Contas	
Nubank	Conta corrente R\$...
Itaú	Poupança R\$...
Dinheiro	Em espécie R\$...
Estado vazio possível:	
Nenhuma conta encontrada	
Início	Saldo
Análises	Trans.

9.7 Análises

Arquivo: app/screens/Analytics/AnalyticsScreen.tsx

Objetivo: mostrar indicadores e gráficos/visualizações de entradas, saídas e economia.

Análises		
[Entradas]	[Saídas]	[Economia]
R\$...	R\$...	R\$...
Entradas vs. Saídas		
Jan	#### income	### expense

```

| Fev  ##### income  ### expense |
| Mar  ##### income  ### expense |
| Abr  #####         ##          |
| Mai  #####         ##          |
+-----+
| Maiores gastos |
| Mercado        37% |
| Transporte      22% |
| Alimentação    18% |
+-----+
| Início | Saldo | Análises | Trans. |

```

9.8 Transações

Arquivo: app/screens/Transactions/TransacionsScreen.tsx

Objetivo: listar e buscar transações, além de abrir o cadastro de nova transação.

```

+-----+
| Transações [ + ] |
+-----+
| [ Buscar transação... x ] |
+-----+
| - Salário Renda +R$... |
| - Mercado Extra Mercado -R$... |
| - Uber Transporte -R$... |
| - Netflix Entreten. -R$... |
| Estado vazio possível: |
| Nenhuma transação encontrada |
+-----+
| Início | Saldo | Análises | Trans. |

```

9.9 Nova transação

Arquivo: app/screens/Transactions/AddTransactionScreen.tsx

Objetivo: cadastrar entrada ou saída financeira.

```

+-----+
| ← Nova Transação |
+-----+
| Nome da Transação |
| [ Ex: Compra no Mercado ] |
| Tipo de Transação |
| [ Entrada ] [ Saída ] |
| Valor |
| R$ [ 0,00 ] |
| Data |
| [ YYYY-MM-DD ] |
| Categoria |
| [ Mercado v ] |
+-----+
| Resumo |
| Transação: ... |
| Data: ... |
| Categoria: ... |
| Tipo: ... |
| Valor: ... |
+-----+
| [ Cancelar ] [ Adicionar ] |
+-----+

```

9.10 Perfil

Arquivo: app/screens/Profile/ProfileScreen.tsx

Objetivo: mostrar dados do usuário, estatísticas rápidas, plano e atalhos de configuração.

```

+-----+
| Perfil config |
+-----+

```

	[avatar/câmera]	
	Nome do usuário	
	e-mail	
	Plano Básico	
+-----+		
	47 6 3	
	Transações Categorias Contas	
+-----+		
	Editar perfil	>
	Segurança	>
	Configurações do app	>
	Ajuda	>
+-----+		
	[Sair]	
+-----+		
	Início Saldo Análises Trans.	

9.11 Editar perfil

Arquivo: app/screens/Profile/EditProfileScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

+-----+		
	← Editar Perfil	
+-----+		
	Implemente o conteúdo desta tela	
	aqui	
+-----+		

9.12 Segurança

Arquivo: app/screens/Profile/SecurityScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

+-----+		
	← Segurança	
+-----+		
	Implemente o conteúdo desta tela	
	aqui	
+-----+		

9.13 Configurações do app

Arquivo: app/screens/Profile/AppSettingScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

+-----+		
	← Configurações	
+-----+		
	Implemente o conteúdo desta tela	
	aqui	
+-----+		

9.14 Ajuda

Arquivo: app/screens/Profile/HelpScreen.tsx

Estado atual: tela placeholder com cabeçalho e mensagem para implementação.

+-----+		
	← Ajuda	
+-----+		
	Implemente o conteúdo desta tela	
	aqui	
+-----+		

10. Fluxos principais

10.1 Login

Usuário abre app
-> AuthContext tenta restaurar tokens do AsyncStorage
-> Se não houver sessão: Login
-> POST /auth/login
-> Backend valida e-mail/senha
-> Backend retorna user + accessToken + refreshToken
-> App salva no AsyncStorage
-> RootNavigator mostra MainTabNavigator

10.2 Renovação de sessão

App restaura accessToken e refreshToken
-> GET /auth/me
-> Se accessToken estiver inválido/expirado
-> POST /auth/refresh
-> Backend valida refreshToken salvo no usuário
-> Backend remove token antigo e gera token novo
-> App salva novos tokens

10.3 Criação de transação

Usuário abre Transações
-> Toca em +
-> Preenche nome, tipo, valor, data e categoria
-> POST /data/transactions com Bearer token
-> Backend valida campos obrigatórios
-> Backend cria transação em memória
-> App recebe a transação criada

11. Pontos fortes

- Separação clara entre frontend, backend, navegação, tema, contexto e utilitários.
- Autenticação com fluxo robusto para login, cadastro, refresh token e logout.
- Uso de hash bcrypt e remoção de campos sensíveis no JSON do usuário.
- Interface mobile consistente, com tema próprio e componentes reutilizáveis.
- Navegação bem estruturada entre autenticação, abas principais e stacks internos.
- Tipagem de rotas no frontend, reduzindo erros de navegação.

12. Lacunas e riscos atuais

Área	Observação	Impacto
Dados financeiros	Transações e contas estão em memória no backend.	Dados são perdidos ao reiniciar o servidor e não são separados por usuário.
URL da API	IP local fixo em API_BASE_URL.	App pode falhar em outro ambiente/rede.
Telas de perfil	Editar perfil, segurança, configurações e ajuda ainda são placeholders.	Funcionalidades de conta ficam incompletas.
Login social	Backend aceita provider/providerId, mas não valida token com Google/Facebook/X.	Risco de autenticação social falsa se exposto em produção.
README	Ainda é o README padrão do Expo.	Falta documentação de instalação real do frontend/backend.
Testes	Não foram identificados testes automatizados.	Maior risco de regressões em autenticação e finanças.

Area	Observação	Impacto
Modelo financeiro	Não há schemas MongoDB para transações, contas e categorias.	Escalabilidade e multiusuário ainda não estão resolvidos.

13. Recomendações técnicas

1. Persistir transações, contas e categorias no MongoDB com relacionamento por user Id.
2. Substituir o IP fixo da API por variável de ambiente Expo ou configuração por ambiente.
3. Implementar as telas placeholder do perfil.
4. Validar login social no backend usando tokens reais dos provedores.
5. Criar testes para autenticação, refresh token, rotas protegidas e criação/listagem de transações.
6. Atualizar o README com instruções reais de setup: frontend, backend, MongoDB e variáveis de ambiente.
7. Adicionar tratamento de loading/erro padronizado nas telas que consomem API.
8. Criar modelo de domínio financeiro: usuário -> contas -> transações -> categorias -> resumo.

14. Scripts úteis

Frontend:

```
npm install
npm run start
npm run android
npm run ios
npm run web
npm run lint
```

Backend:

```
cd backend
npm install
npm run dev
npm start
```

Variáveis esperadas no backend:

```
MONGODB_URI=mongodb+srv://...
JWT_SECRET=...
JWT_REFRESH_SECRET=...
PORT=4000
```

15. Conclusão

O FinTrackApp já tem uma base sólida de aplicativo mobile financeiro: frontend em Expo/React Native, backend Express, autenticação com JWT, tema visual consistente e navegação organizada. O próximo passo mais importante é transformar os dados financeiros de mock/memória em dados persistidos e isolados por usuário no MongoDB. Depois disso, o app passa de protótipo funcional para uma base mais próxima de produção.